00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D

ALGORİTMA VE PROGRAMLAMA – II

HAFTA 08

Dinamik Diziler ve Bağlı Listeler

Bellekte Esnek Yapılar

MODÜL III · DİNAMİK VERİ VE HIZLI ERİŞİM

Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Bahar

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Doğruluk ve Ölçüm	**H2** Asimptotik Analiz	**H3** Özyineleme ve Geri İzleme	**H4** Arama Algoritmaları	**H5** Temel Sıralama	**H6** Böl ve Fethet	**H7** Öbek ve Doğrusal Sıralama
0x00	0x01	0x02	0x03	0x04	0x05	0x06
H8 **Dinamik Dizi ve Bağlı Liste**	**H9** Yığın ve Kuyruk	**H10** Karma Tabloları	**H11** Graflar: BFS ve DFS	**H12** Açgözlü Algoritmalar	**H13** Dinamik Programlama	**H14** Sentez ve Final Projesi
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 (teslim: H8 başı) · Final projesi: H14

Geçen haftadan ve AP-1'den: üç soru

- **S1 (AP-1 H12).** `dizi = realloc(dizi, 2 * kap * sizeof *dizi);` satırının tehlikesi nedir?
- **S2 (AP-1 H8).** `takas(int a, int b)` çağrısından sonra `main`'deki a ve b neden değişmez? Çağıranın **işaretçisini** değiştiren fonksiyonun parametresi hangi türdedir?
- **S3 (H7).** Öbekte ebeveyn $\lfloor (i-1)/2 \rfloor$ indisinde, O(1). Tek bağlı listede bir düğümün **öncülünü** bulmak ne kadar sürer?

Büyük Soru: ortaya eklemede liste her zaman daha mı hızlı?

72,9×

C, $n = 5 \cdot 10^4$: listenin süresi / dizinin süresi (tek ölçüm)

35,7×

C++, $n = 5 \cdot 10^4$: std::list / std::vector

%99,7

listede sürenin konum bulmaya giden payı (C)

- Bir dizinin ortasına eleman eklemek için bağlı liste her zaman daha mı hızlıdır? İş yükü: sıralı yapıya doğrusal aramayla ekle, sonra rastgele konumlardan sil. Bu makinede yanıt: hayır.



GERÇEK DÜNYADAN

isocpp.org · 2014

“Liste, elbette!” Stroustrup'un sorusu

- C++'ın tasarımcısı **Bjarne Stroustrup**, John Bentley'nin bir alıştırmasını birçok konuşmasında kullandığını yazar: rastgele tamsayıları **sıralı** bir diziye ekle, sonra rastgele konumlardan tek tek sil.
- Çoğu kişinin ilk yanıtı “liste, elbette!”; Stroustrup'a göre bu yanıt **tamamen ve çarpıcı biçimde yanlış**. Gerekçe önbellek: bağlantılar önceden ayrılsa bile listenin dolaşılması performansı öldürür.
- Üç ders: veriyi **gereksiz yere saklamayın**, veriyi **sıkışık** tutun, belleğe **öngörülebilir** biçimde erişin.
- Yazıda ölçüm tablosu ya da grafik yok: bu dersteki bütün süreler **bizim ölçümümüz**. Öğeler çok büyükse listeler eninde sonunda öne geçer.

Stroustrup, B. (2014). Are lists evil? Standard C++ Foundation.

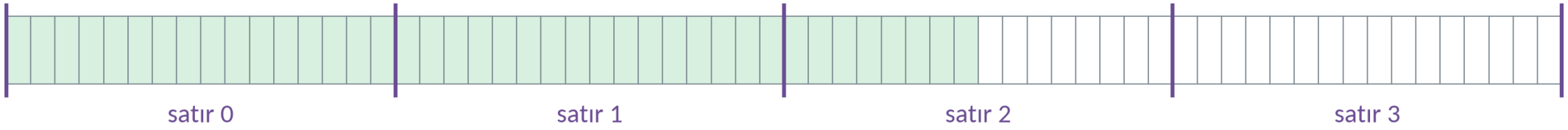
ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

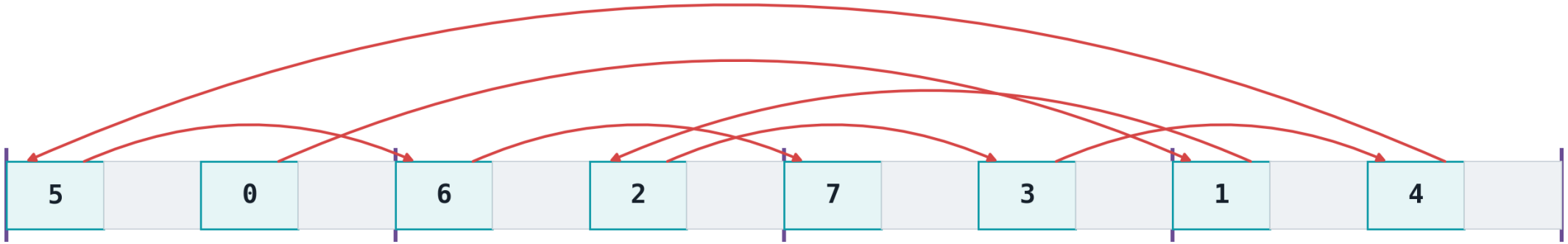
ÖÇ 8.1	Kapasitesini ikiye katlayarak büyüyen dinamik diziyi gerçekler ve sona eklemenin amortize O(1) olduğunu toplam yöntemiyle gösterir.
ÖÇ 8.2	Tek ve çift bağlı listeyi düğüm yapısıyla gerçekler; başa, sona ve araya ekleme ile silme işlemlerini sınır durumlarıyla birlikte doğru kodlar.
ÖÇ 8.3	Liste algoritmalarını (ters çevirme, iki sıralı listeyi birleştirme, ortadaki düğüm, döngü tespiti) iki işaretçi tekniğiyle çözer.
ÖÇ 8.4	Dizi ve bağlı listeyi işlem karmaşıklığı, bellek kullanımı ve önbellek davranışı açısından deneysel olarak karşılaştırır.
ÖÇ 8.5	Bellek sahipliğini ve yaşam döngüsünü yönetir; sızıntı ve sarkan işaretçi hatalarını AddressSanitizer ve Valgrind ile tespit eder.

Bitişik dizi, dağınık düğümler

int dizisi: öğeler bitişik, 4 bayt; bir önbellek satırı (64 B) = 16 öğe → bir kaçırma 16 öğe getirir



bağlı liste: düğüm 16 B (sizeof), öbekte 32 B'lık dilim; bağlantılar satırlar arasında atlar (karışık sıra)

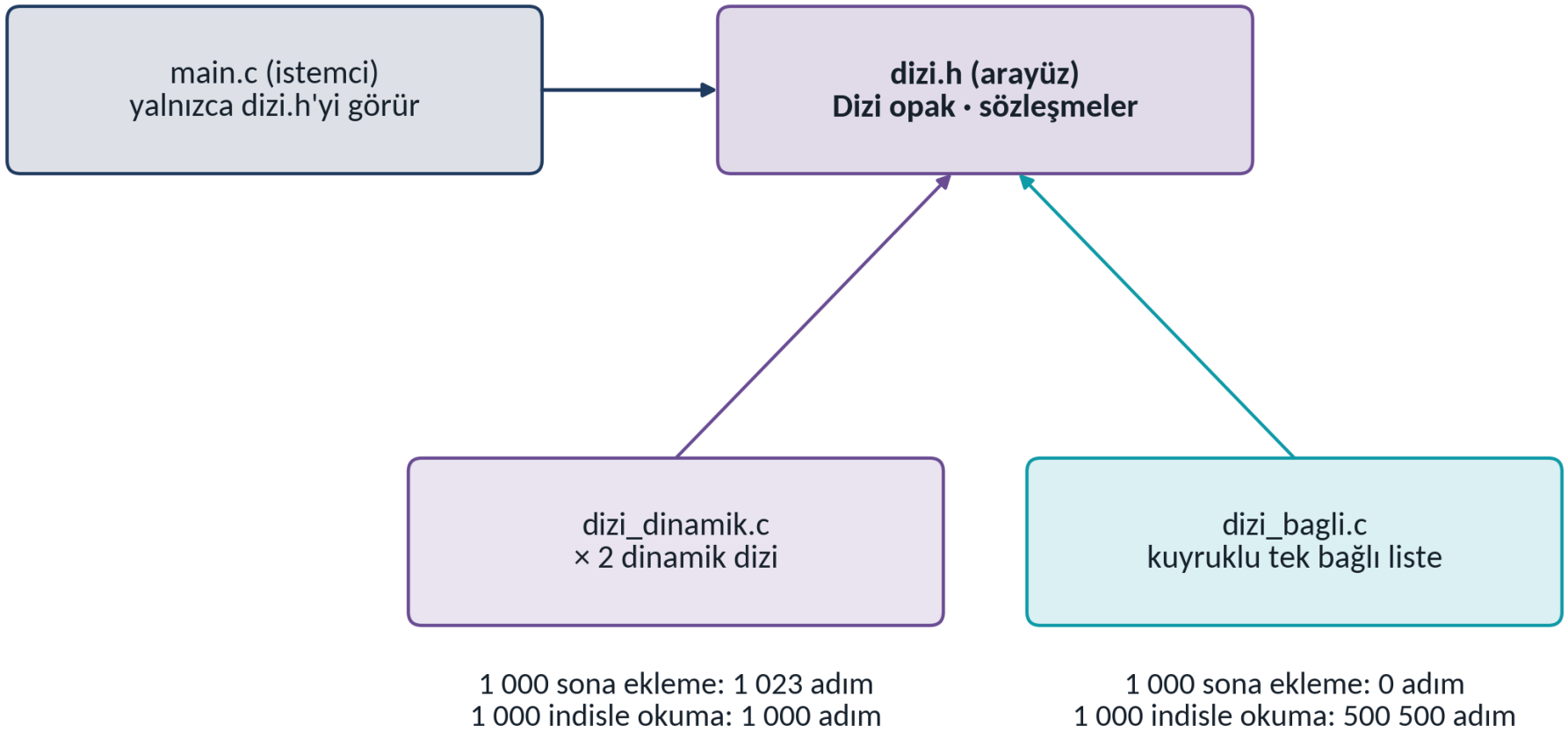


açık mavi: düğüm verisi · gri: ayırıcı başlığı + hizalama · mor çizgi: 64 B önbellek satırı sınırı

bu makinede (C): n = $3 \cdot 10^5$ düğümde dolaşma düğüm başına dizi 0,48 ns, bitişik liste 2,55 ns, karışık liste 146,0 ns

Bu makinede düğüm 16 bayt (sizeof), öbekte 32 baytlık blok: int başına dizi 4, liste 32 bayt · 64 baytlık önbellek satırı

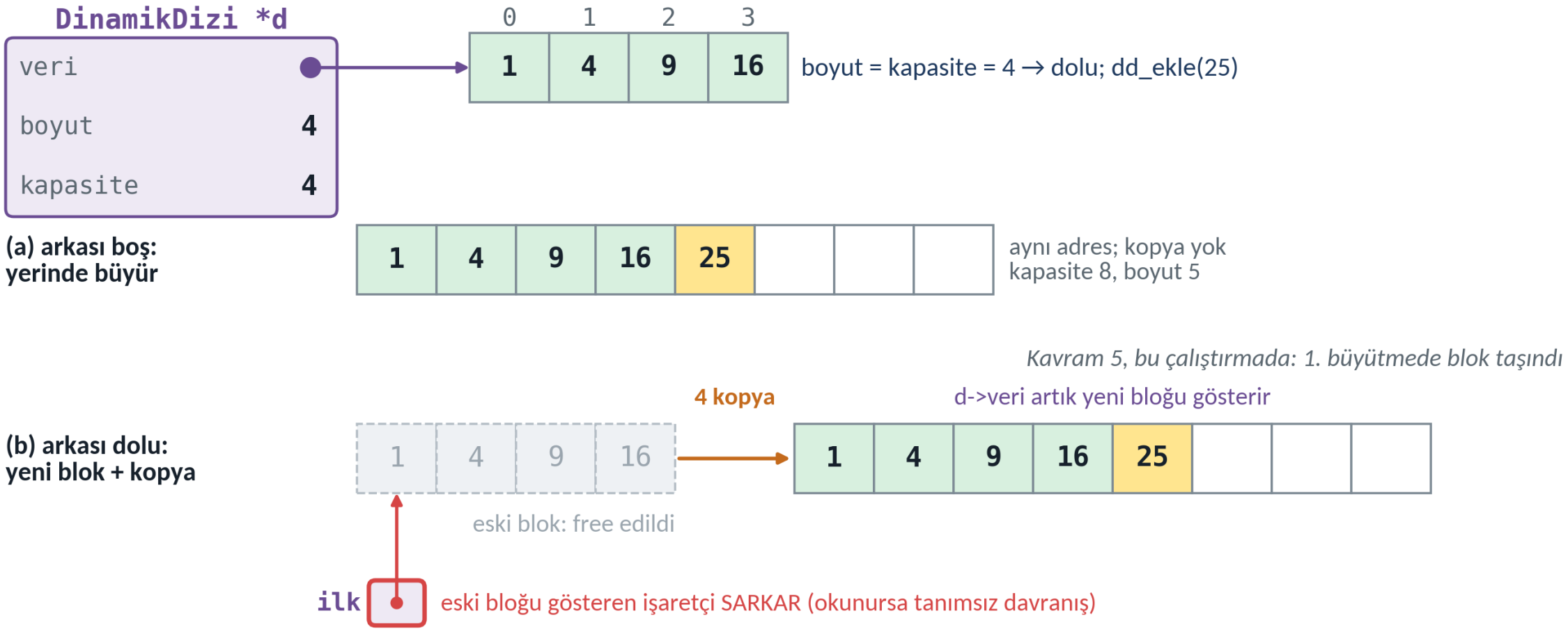
Soyut veri tipi: bir arayüz, iki gerçekleştirim



aynı arayüz, aynı çıktı;
farklı maliyet (Kavram 1)

Aynı main.c: indisle 1 000 okuma dinamik dizide 1 000, listede 500 500 adım (Kavram 1) · maliyet de sözleşmenin parçası

Dinamik dizi: boyut, kapasite ve realloc'un iki yolu



Doluyken ekleme: ya yerinde büyüt ya yeni blok + kopya + eskiyi bırak; taşınınca eski işaretçilerin hepsi (C++'ta yineleyiciler de) geçersiz

TAHMİN ET

Kapasite 32 ve dolu. 33. eklemede kaç öğe kopyalanır?

A 1

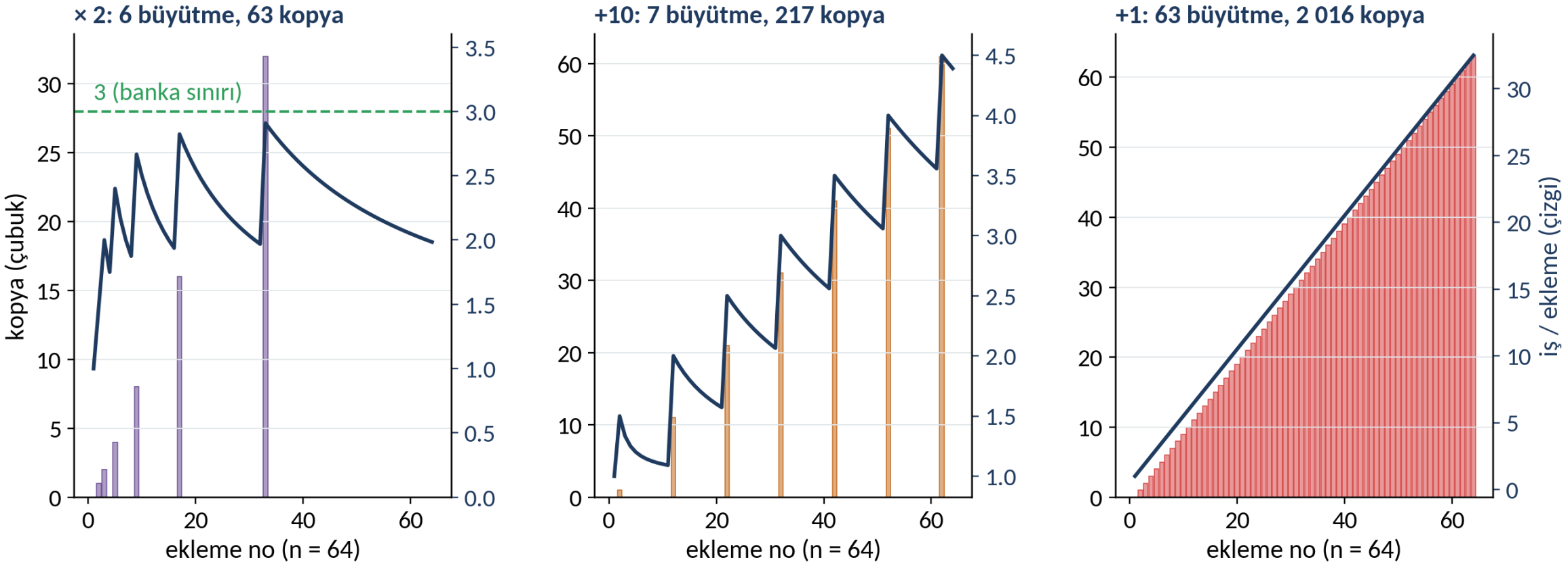
B 16

C 32

D 33

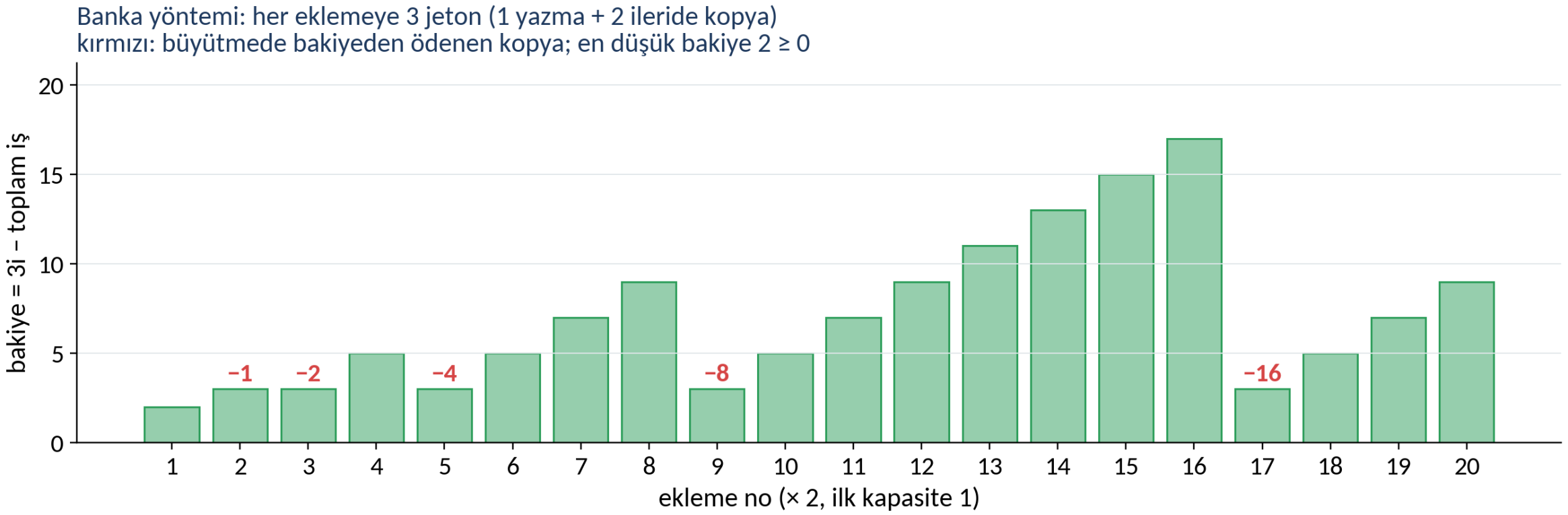
Kopyalama tepeleri seyrek, ortalama sabit

Büyüme zaman çizelgesi: kopyalama tepeleri (çubuk), amortize ortalama (çizgi); doğrusal ölçek



Büyüme zaman çizelgesi · n = 10^5: +1 ile 4 999 950 000, +10 ile 499 960 000, × 1,5 ile 276 492, × 2 ile 131 071 kopya (Örnek 2)

Banka sezgisi: her eklemeye 3 jeton



1 jeton yazmaya, 2 jeton bankaya; büyütmede biriken jetonlar kopyayı öder, bakiye hiç negatif olmaz

ALGORİTMA KİMLİK KARTI

Dinamik dizi: sona ekleme *dynamic array append (push_back)*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Dolunca × 2 büyüt, öğeleri kopyala	İlk kapasite ≥ 1	**O(1)**	**O(1) amortize**	**O(n)**	**O(n) toplam**

Özellikler	İndisle erişim O(1); 10^6 eklemede 1 048 575 kopya (öğe başına 1,049); bedel: boş yuva
Değişmez / doğruluk	boyut ≤ kapasite; toplam kopya 1 + 2 + 4 + … < 2n
Kütüphane karşılığı	C: yok (realloc ile elle) · C++: std::vector::push_back · Python: list.append
Ne zaman kullanılır	Sona eklenip indisle okunan veriler; dizi tabanlı yığın (H9)
Ne zaman kullanılmaz	Tek eklemenin O(n) gecikmesi kabul edilemiyorsa; öğelere dışarıdan işaretçi tutuluyorsa

C: sona ekleme, realloc geçici işaretçiyle

kavram/H08_K01_dizi_adt/dizi_dinamik.c · satır 28–42

C

```c
int dizi_ekle(Dizi *d, int x)
{
    if (d->boyut == d->kapasite) {
        size_t yeni = d->kapasite ? 2 * d->kapasite : 1;
        int *g = realloc(d->veri, yeni * sizeof *g);
        if (g == NULL) {
            return -1;            /* d->veri hâlâ geçerli */
        }
        adim += (long long)d->boyut;   /* model: taşınan öğe */
        d->veri = g;
        d->kapasite = yeni;
    }
    d->veri[d->boyut++] = x;
    return 0;
}
```

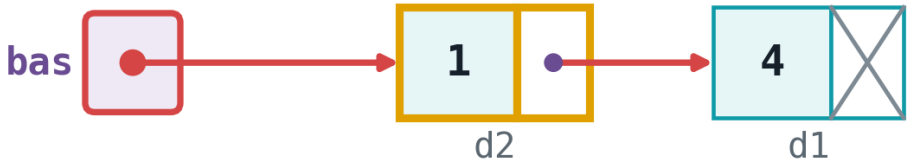
- Doluysa kapasiteyi ikiye katla (0 → 1).
- `realloc` sonucu **önce geçici** işaretçiye: başarısızlıkta eski blok geçerli kalır.
- Sayaç: taşınan öğe kadar adım (model); Örnek 1'in `dd_ekle`'si de aynı.

Bağlı liste: başa ekleme iki bağlantı

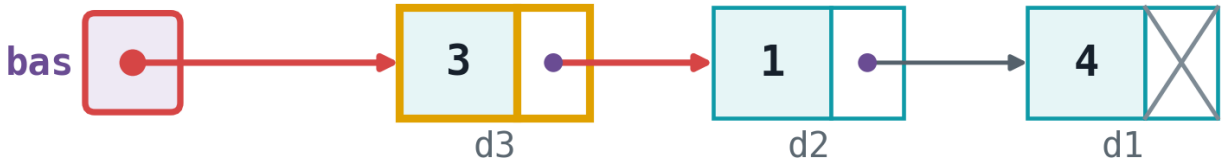
1) basa_ekle(4): yeni düğümün sonraki'si eski başa, bas yeni düğüme



2) basa_ekle(1): yeni düğümün sonraki'si eski başa, bas yeni düğüme



3) basa_ekle(3): yeni düğümün sonraki'si eski başa, bas yeni düğüme



kırmızı: bu adımda değişen bağlantı
sarı kenar: yeni düğüm
çarpı: NULL

Örnek 3: basa_ekle 4, 1, 3; yeni düğümün sonraki'si eski başa, bas yeni düğüme · Örnek 4: kuyruk işaretçisiyle sona ekleme O(1)

TAHMİN ET

[3 1 9 4] listesine konuma_ekle(0, 7). İşlemden sonra baş nereyi gösterir?

A 3 (baş değişmez)

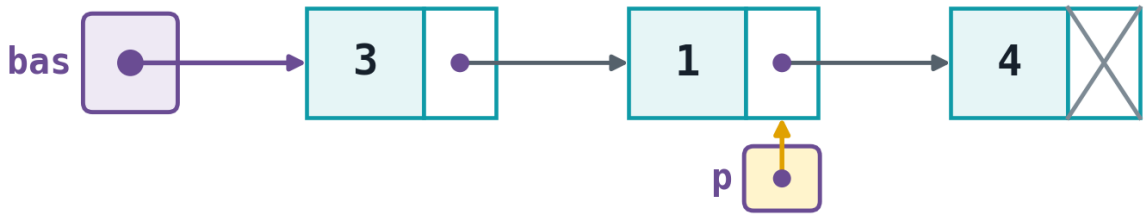
B 7 (yeni düğüm)

C NULL

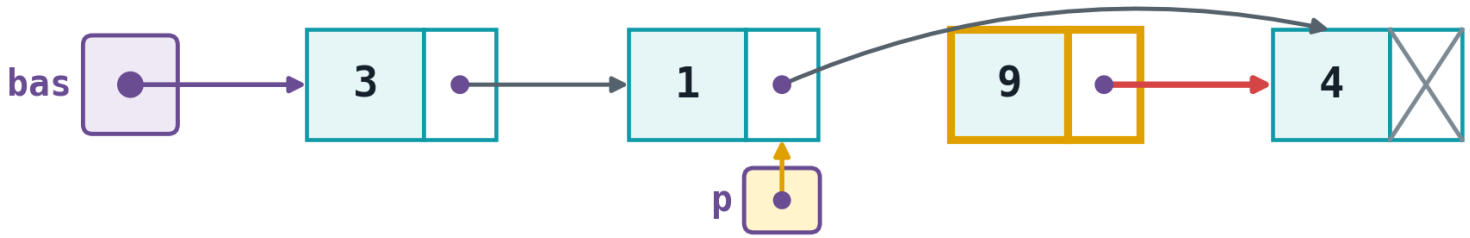
D 9

Araya ekleme: sıra önemli

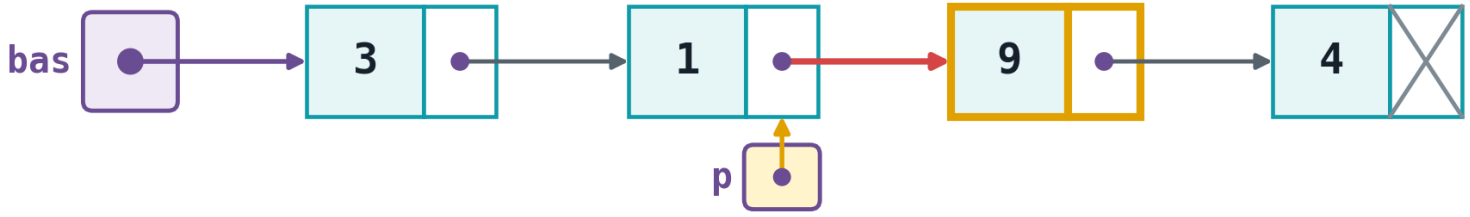
önce: p = &bas; 2 kez p = &(*p)->sonraki → p, 1'in sonraki ALANINI gösterir



1) d->sonraki = *p; yeni düğüm (9) 4'ü gösterir; liste değişmedi



2) *p = d; 1'in sonraki alanı yeni düğümü gösterir: [3 → 1 → 9 → 4]

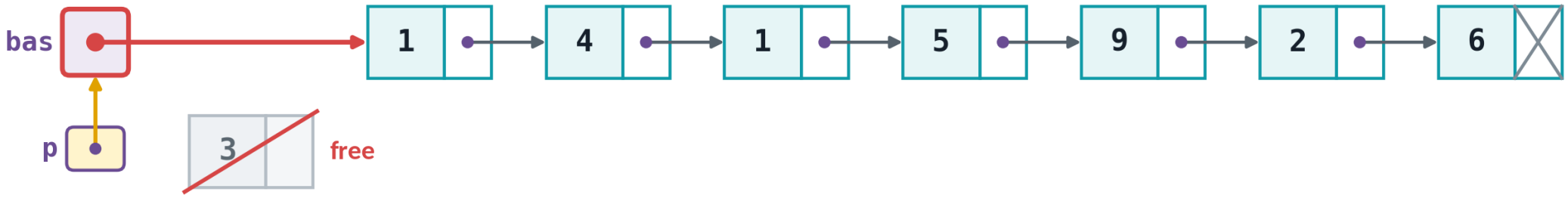


Kavram 2: konuma_ekle(2, 9)
ziyaret 2, bağlantı yazma 2
(iki atama; sıra önemli)

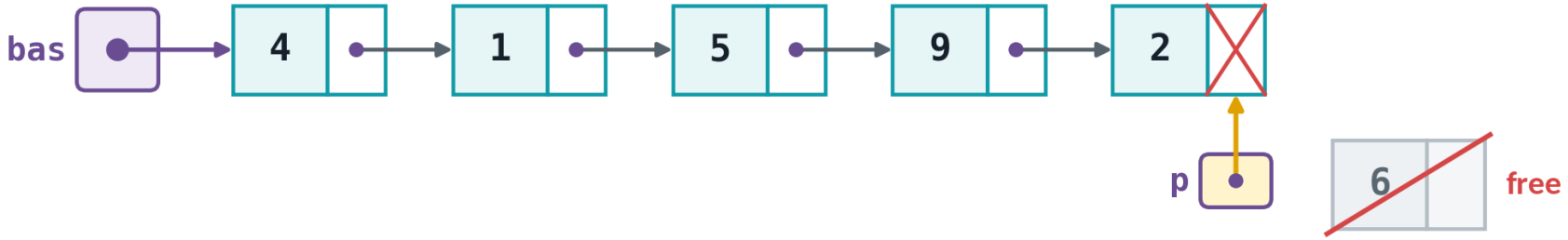
Öncüle kadar yürü (Θ(konum)); önce yeni düğümün sonraki'si, sonra öncülün sonraki'si; ters sırada kalan kaybolur

Dugum **p bir bağlantıyı gösterir

sil 3 (baş): p = &bas; p, bas DEĞİŞKENİNİ gösterir (ziyaret = 1)



sil 6 (son): p, 2'nin sonraki ALANINI gösterir (ziyaret = 6)




```
iki durumda da TEK satır: *p = silinen->sonraki;  (baş için ayrı dal yok)
```

*p = &bas ya da p = &d->sonraki: baş da bir bağlantıdır; silme her durumda tek satır: *p = silinen->sonraki*

Özel durumu ortadan kaldırmak

Özel durumlu: onceki + if

H08_K03_ozel_durumsuz_silme.c · satır 21–34 — C

```c
21  static void cikar_ozel(Liste *l, Dugum *hedef)
22  {
23      Dugum *onceki = NULL;
24      Dugum *p = l->bas;
25      while (p != hedef) {
26          onceki = p;
27          p = p->sonraki;
28      }
29      if (onceki == NULL) {
30          l->bas = hedef->sonraki;
31      } else {
32          onceki->sonraki = hedef->sonraki;
33      }
34  }
```

Dugum **: dal yok

H08_K03_ozel_durumsuz_silme.c · satır 37–44 — C

```c
37  static void cikar_pp(Liste *l, Dugum *hedef)
38  {
39      Dugum **p = &l->bas;
40      while (*p != hedef) {
41          p = &(*p)->sonraki;
42      }
43      *p = hedef->sonraki;
44  }
```

Torvalds, TED 2016: ikinci sürümde **if yok**; problemi farklı görünce özel durum olağan duruma dönüşür (Kavram 3)

Üç dilde: özel durumsuz silme

H08_05_degere_gore_silme.c — C

```c
Dugum **p = bas;
while (*p != NULL && !aranan_mi(*p, x)) {
    p = &(*p)->sonraki;
}
if (*p == NULL) {
    return false;
}
Dugum *silinen = *p;
*p = silinen->sonraki;  /* baş da aynı satır */
```

H08_05_degere_gore_silme.cpp — C++

```cpp
Dugum** p = &bas;
while (*p != nullptr && !aranan_mi(*p, x)) {
    p = &(*p)->sonraki;
}
if (*p == nullptr) {
    return false;
}
Dugum* silinen = *p;
*p = silinen->sonraki;  // baş da aynı satır
```

H08_05_degere_gore_silme.py — Python

```python
kukla = Dugum(0)
kukla.sonraki = bas
onceki = kukla
while onceki.sonraki is not None and not aranan_mi(onceki.sonraki, x):
    onceki = onceki.sonraki
if onceki.sonraki is None:
    return kukla.sonraki, False
silinen = onceki.sonraki
onceki.sonraki = silinen.sonraki  # baş da aynı
```

ALGORİTMA KİMLİK KARTI

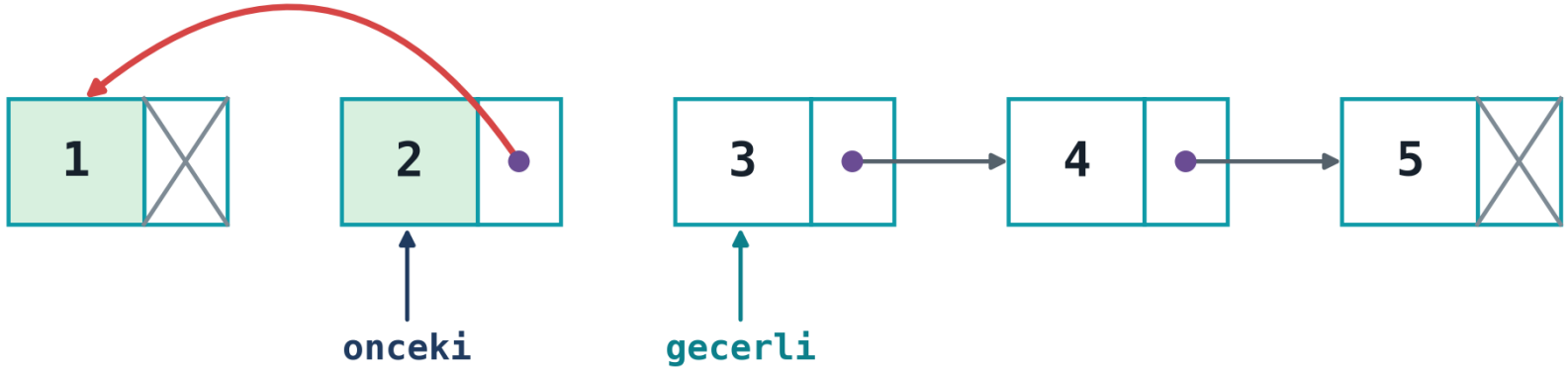
Bağlı liste: başa ekleme ve arama *singly linked list: push front / linear search*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Bağlantı kur; baştan doğrusal dolaş	Yok (boş liste: bas = NULL)	O(1) / O(1)	O(1) / Θ(n)	O(1) / Θ(n)	Düğüm başına O(1)

Özellikler	Rastgele erişim yok; başarılı aramada ortalama 500,50 ziyaret (n = 1000); kuyruksuz 1000 sona ekleme 499 500 ziyaret
Değişmez / doğruluk	Bakılan ilk i düğümün hiçbiri x değildir
Kütüphane karşılığı	C: yok (elle) · C++: std::forward_list · Python: collections.deque
Ne zaman kullanılır	Sık başa ekleme/çıkarma; düğüm adresi sabit kalmalı: yığın, kuyruk, zincir
Ne zaman kullanılmaz	İndisle erişim ya da sık arama; küçük öğelerde bellek ve önbellek bedeli yüksek

Ters çevirme: kaydet, çevir, ilerle

3/6 · adım 2: sonraki = 3; 2 → 1; ilerle




```
ters: [2 -> 1 -> NULL]    kalan: [3 -> 4 -> 5 -> NULL]
```

- **Değişmez:** `onceki` ters çevrilmiş önekin başı (yeşil), `gecerli` işlenmemiş sonekin başı.
- Her turda üç adım: `sonraki = gecerli->sonraki` (kaydet), `gecerli->sonraki = onceki` (çevir), ikisini ilerlet.

TAHMİN ET

Bir sonraki adımda (adım 3) hangi bağlantı çevrilir ve “kalan” ne olur?

A 3 → NULL; kalan [5 -> NULL]

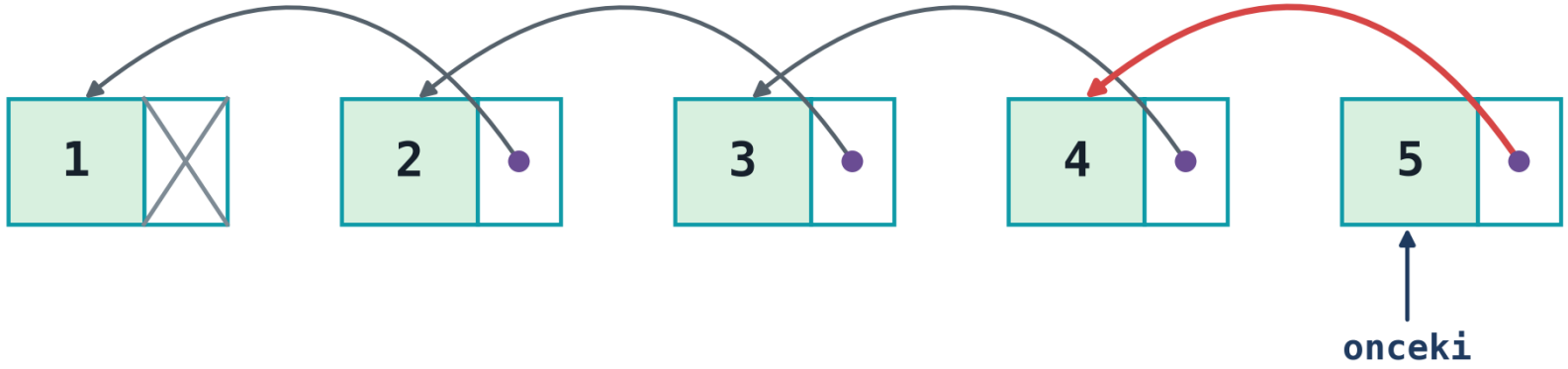
B 2 → 3; kalan [3 -> 4 -> 5 -> NULL]

C 3 → 4; kalan [NULL]

D 3 → 2; kalan [4 -> 5 -> NULL]

Bitti: onceki yeni baş

6/6 · adım 5: sonraki = NULL; 5 → 4; ilerle → dur




```
ters: [5 -> 4 -> 3 -> 2 -> 1 -> NULL]    kalan: [NULL]
```

- sonuç: [5 -> 4 -> 3 -> 2 -> 1 -> NULL] bağlantı yazma = 5 (n)
- Özyinelemeli sürüm: `bağlantı yazma = 8 (2(n - 1)), en büyük derinlik = 5 (n)`: O(n) yığın.

Ters çevirme: sözde kod ve C

```
algoritma TERS-ÇEVİR(bas)
  onceki ← NULL
  gecerli ← bas
  iken gecerli ≠ NULL
    sonraki ← gecerli.sonraki
    gecerli.sonraki ← onceki
    onceki ← gecerli
    gecerli ← sonraki
  döndür onceki
```

c/H08_07_ters_cevirme.c · satır 91–103

C

```c
static Dugum *ters_cevir(Dugum *bas)
{
    Dugum *onceki = NULL;
    Dugum *gecerli = bas;
    while (gecerli != NULL) {
        Dugum *sonraki = gecerli->sonraki; /* kaydet */
        gecerli->sonraki = onceki;         /* çevir */
        baglanti_sayisi++;
        onceki = gecerli;                  /* ilerle */
        gecerli = sonraki;
    }
    return onceki;
}
```

Θ(n) zaman · O(1) ek bellek · n bağlantı yazma · yeni düğüm yok

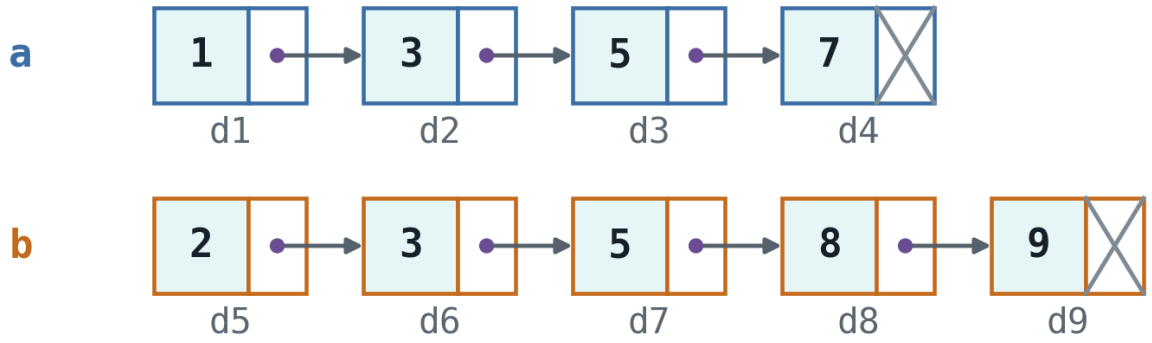
ALGORİTMA KİMLİK KARTI

Listeyi ters çevirme *linked list reversal*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Bağlantıları yerinde yeniden kur	Yok	Θ(n)	Θ(n)	Θ(n)	O(1)

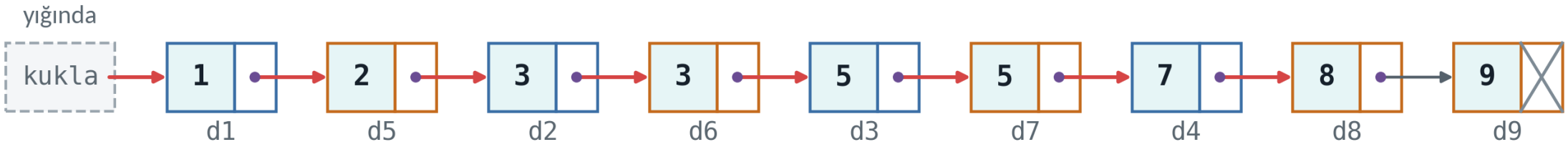
Özellikler	onceki–gecerli–sonraki üçlüsü; her düğüme bir bağlantı yazılır; özyinelemeli sürüm 2(n − 1) yazma, n derinlik
Değişmez / doğruluk	onceki ters önekin, gecerli kalan sonekin başı
Kütüphane karşılığı	C: yok (elle) · C++: std::forward_list::reverse · Python: list.reverse() (dizide)
Ne zaman kullanılır	Başa ekleyerek kurulan listeyi giriş sırasına çevirmek; palindrom denetimi
Ne zaman kullanılmaz	Özyinelemeli sürüm büyük n'de: Python sınırı ≈ 1000, C'de bu makinede 10^6 yığın taştı

İki sıralı listeyi birleştirme: kukla düğüm



önce: iki sıralı liste
(kimlik dK: a = d1–d4, b = d5–d9)

sonra: 7 karşılaştırma adımında küçük baş son'un ardına bağlandı; kalan kuyruk (d8 → d9) TEK bağlantıyla eklendi



mavi kenar: a'dan · turuncu kenar: b'den · kırmızı: kurulan bağlantı (8) · yeni düğüm: 0

a = [1 -> 3 -> 5 -> 7 -> NULL] · b = [2 -> 3 -> 5 -> 8 -> 9 -> NULL] · 7 seçim + kalan kuyruk tek bağlantı · yeni düğüm 0

Üç dilde: kukla düğümle birleştirme döngüsü

H08_08_sirali_birlestirme.c — C

```c
if (a->veri <= b->veri) {
    son->sonraki = a;
    a = a->sonraki;
} else {
    son->sonraki = b;
    b = b->sonraki;
}
baglanti_sayisi++;
son = son->sonraki;
```

H08_08_sirali_birlestirme.cpp — C++

```cpp
if (a->veri <= b->veri) {
    son->sonraki = a;
    a = a->sonraki;
} else {
    son->sonraki = b;
    b = b->sonraki;
}
baglanti_sayisi++;
son = son->sonraki;
```

H08_08_sirali_birlestirme.py — Python

```python
if a.veri <= b.veri:
    son.sonraki = a
    a = a.sonraki
else:
    son.sonraki = b
    b = b.sonraki
baglanti_sayisi += 1
son = son.sonraki
```

ALGORİTMA KİMLİK KARTI

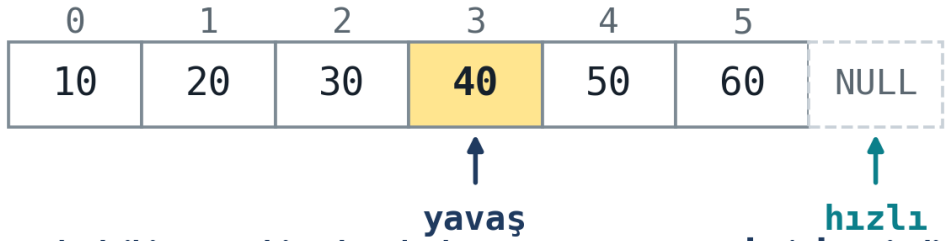
İki sıralı listeyi birleştirme *merging two sorted linked lists*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
İki işaretçi + kukla düğüm	İki liste de artan sıralı	$\Theta(n + m)$	$\Theta(n + m)$	$\Theta(n + m)$	$O(1)$

Özellikler	Yeniden bağlama (yeni düğüm 0); kararlı; n = m = 10^5'te 199 746 karşılaştırma (≤ n + m − 1)
Değişmez / doğruluk	kukla.sonraki'den son'a uzanan zincir alınanların kararlı sıralı birleşimidir
Kütüphane karşılığı	C: yok (elle) · C++: std::list::merge · Python: heapq.merge
Ne zaman kullanılır	Listede birleştirmeli sıralama; iki sıralı kaydın birleştirilmesi (Mini-OBS 2 v2.7)
Ne zaman kullanılmaz	Listeler sıralı değilse; girdi listeleri sonradan ayrı ayrı gerekiyorsa

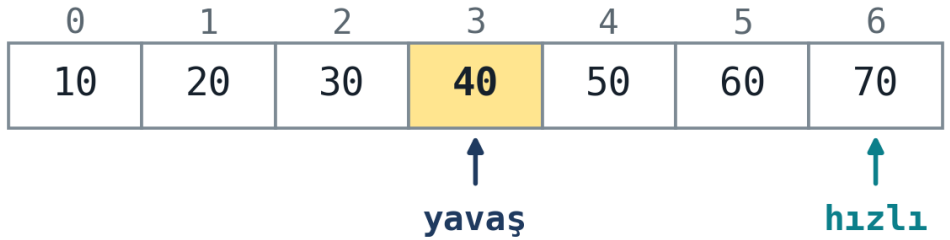
Yavaş–hızlı ve sondan k'ıncı: iki işaretçi

ortadaki, n = 6: hızlı iki, yavaş bir adım; hızlı sona varınca yavaş ⌊n/2⌋ = 3 indisinde (çift n: İKİNCİ orta)



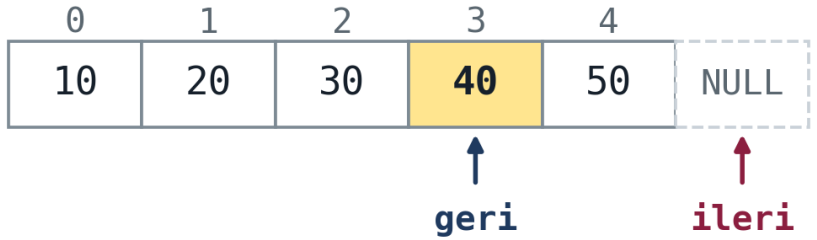

```
(yavaş, hızlı):
(0,0) (1,2) (2,4) (3,NULL)
```

ortadaki, n = 7: hızlı iki, yavaş bir adım; hızlı sona varınca yavaş ⌊n/2⌋ = 3 indisinde




```
(yavaş, hızlı):
(0,0) (1,2) (2,4) (3,6)
```

sondan k, n = 5, k = 2: önce ileri k adım önden; sonra ikisi birlikte; ileri NULL olunca geri hedefte (indis 3)

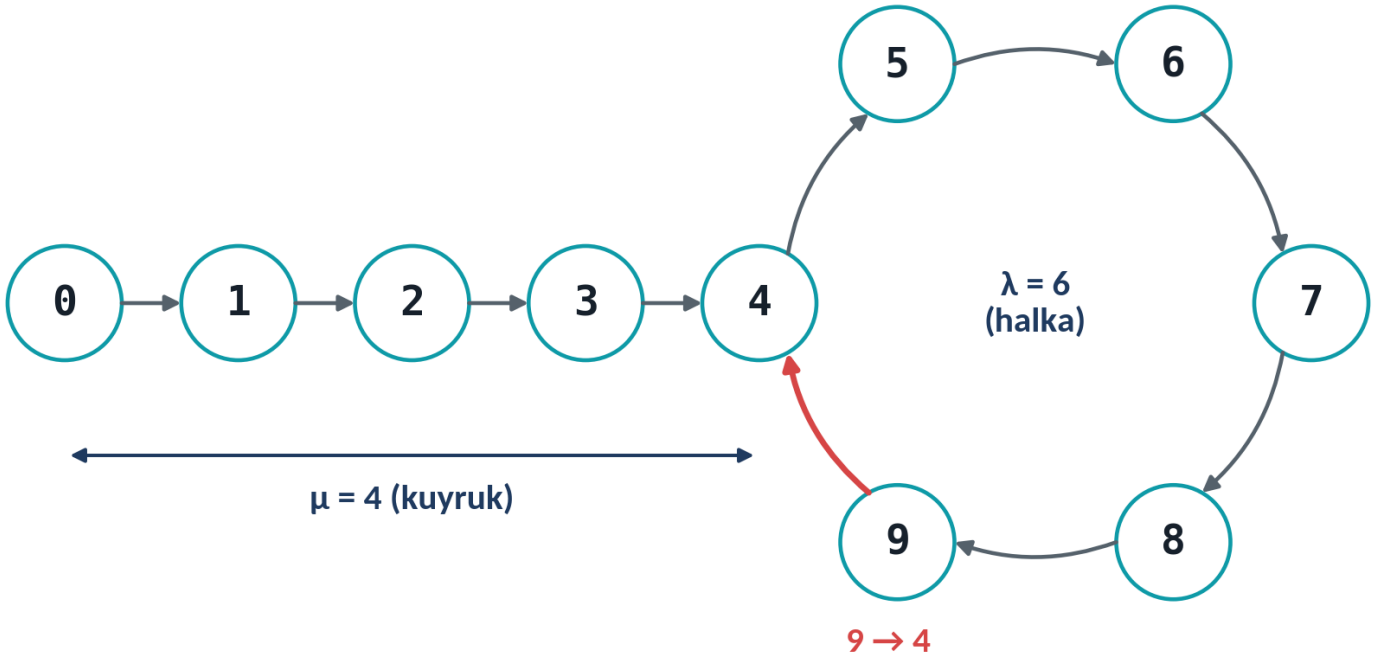



```
(geri, ileri):
(–,0) (–,1) (–,2) (1,3) (2,4) (3,NULL)
```

Örnek 9: hızlı iki, yavaş bir adım; çift n'de ikinci orta · sondan k: ileri k adım önden, sonra birlikte (k < 1 ya da k > n: NULL)

Floyd: kaplumbağa ve tavşan nerede buluşur?

n = 10, son düğüm 4 konumuna bağlı (Örnek 10)



evre 1 (kaplumbağa 1, tavşan 2):

```
adım 0: K = 0, T = 0
adım 1: K = 1, T = 2
adım 2: K = 2, T = 4
adım 3: K = 3, T = 6
adım 4: K = ?, T = ?
…
```

Sizce hangi düğümde buluşurlar?

Kaplumbağa bir, tavşan iki adım; döngü varsa tavşan her adımda bir düğüm yaklaşır

TAHMİN ET

n = 10, son düğüm 4 konumuna bağlı (μ = 4, λ = 6). Kaplumbağa ile tavşan hangi düğümde buluşur?

A 6

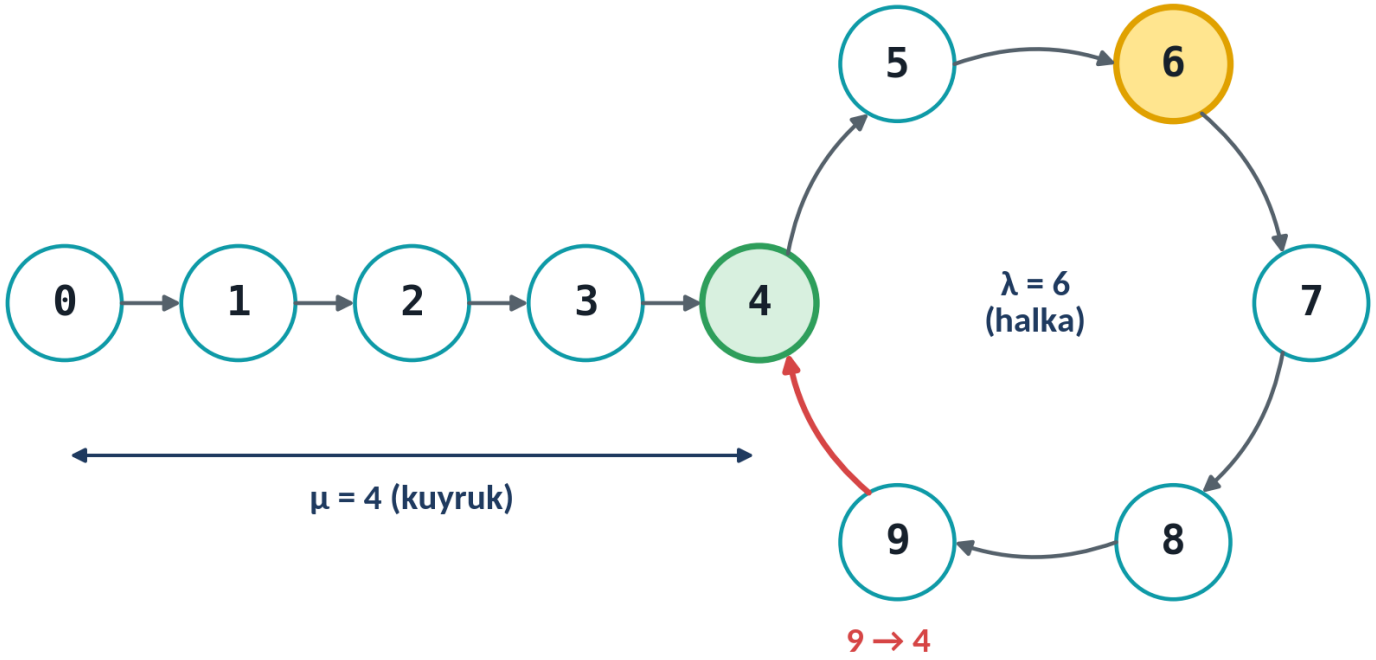
B 4

C 9

D 0

Floyd: buluşma, sonra baştan ve buluşmadan birer adım

n = 10, son düğüm 4 konumuna bağlı (Örnek 10)
sarı: buluşma 6 · yeşil: döngü başı 4



bağıntı: buluşmada K 6 adım attı; 6 mod λ = 0 → buluşmadan μ adım ileri = döngü başı

evre 1 (kaplumbağa 1, tavşan 2):

```
adım 0: K = 0, T = 0
adım 1: K = 1, T = 2
adım 2: K = 2, T = 4
adım 3: K = 3, T = 6
adım 4: K = 4, T = 8
adım 5: K = 5, T = 4
adım 6: K = 6, T = 6
buluşma: 6 (6 adım)
```

evre 2 (K baştan, T buluşmadan; birer):

```
adım 1: K = 1, T = 7
adım 2: K = 2, T = 8
adım 3: K = 3, T = 9
adım 4: K = 4, T = 4
başlangıç: 4 (μ = 4 adım)
```

Örnek 10 · evre 1: 6 adımda 6'da buluşma · evre 2: 4 adımda başlangıç 4 · evre 3: λ = 6 · O(1) ek bellek

Floyd: üç evre, sabit bellek

```
algoritma FLOYD(bas)
  k ← bas;  t ← bas
  iken t ≠ NULL ve
       t.sonraki ≠ NULL
    k ← k.sonraki
    t ← t.sonraki.sonraki
    eğer k = t ise dur
  ▷ evre 2: k ← bas; birer adım
  ▷ evre 3: t'den t'ye say: λ
```

kavram/H08_K04_floyd_bagintisi.c · satır 20–33

C

```c
static Dugum *bulusma(Dugum *bas)
{
    Dugum *kap = bas, *tav = bas;
    adim = 0;
    while (tav != NULL && tav->sonraki != NULL) {
        kap = kap->sonraki;
        tav = tav->sonraki->sonraki;
        adim++;
        if (kap == tav) {
            return kap;
        }
    }
    return NULL;
}
```

Evre 1 ≤ μ + λ adım · evre 2 μ adım · Θ(n) zaman · O(1) bellek; C/C++'ta `==`, Python'da `is`

ALGORİTMA KİMLİK KARTI

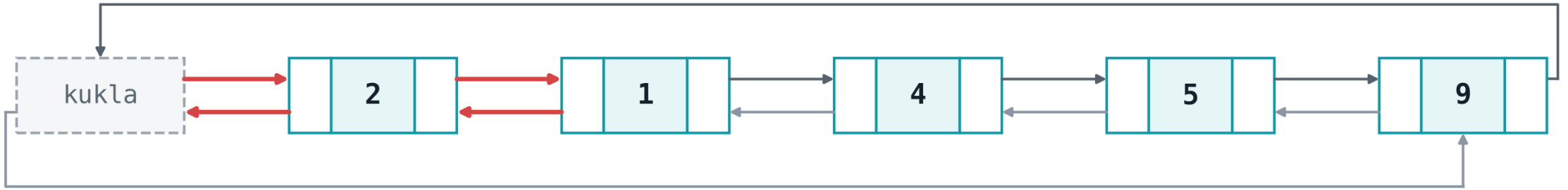
Floyd döngü tespiti *Floyd's cycle detection (tortoise and hare)*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Kaplumbağa 1, tavşan 2 adım	Her düğümün en çok bir sonraki'si	O(1)	Θ(n)	Θ(n)	O(1)

Özellikler	Evre 1 buluşma, evre 2 başlangıç μ, evre 3 uzunluk λ; 1000 rastgele listede 1000/1000 doğru
Değişmez / doğruluk	Buluşmadaki adım sayısı λ'nın katı: baştan ve buluşmadan μ adım aynı düğüme varır
Kütüphane karşılığı	C: yok · C++: yok · Python: yok
Ne zaman kullanılır	Döngüyü sabit bellekle denetlemek (yok etmeden önce); x ← f(x) dizisinin dönemi
Ne zaman kullanılmaz	Ziyaret bayrağı ya da küme kullanılabiliyorsa daha basit (ama O(n) bellek)

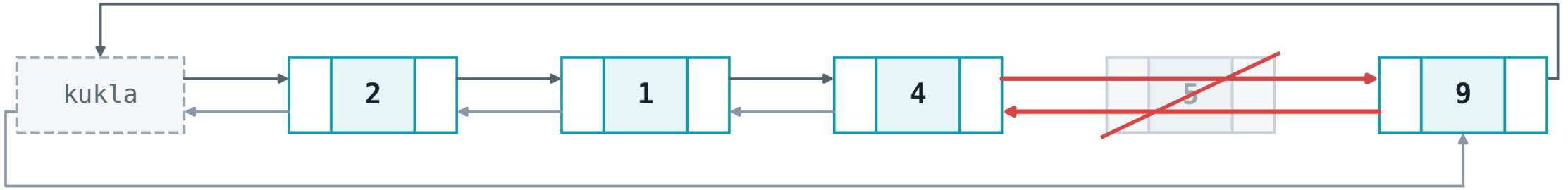
Çift bağlı dairesel liste: adresi bilinen düğüm O(1)

kurma (sona_ekle 1, 4, 5, 9; basa_ekle 2): kırmızı = basa_ekle(2)'nin dört bağlantısı; hiçbiri NULL değil



üst oklar: sonraki · alt oklar: onceki

sil 5 (adresi biliniyor): d->onceki->sonraki = d->sonraki; d->sonraki->onceki = d->onceki



Kukla düğüm boş listede kendini gösterir: hiçbir komşu NULL değil, ekleme 4 bağlantı, silme 2 bağlantı, özel durum yok

Üç dilde: adresi bilinen düğümü O(1) silmek

H08_11_cift_bagli_liste.c — C

```c
    d->onceki->sonraki = d->sonraki;
    d->sonraki->onceki = d->onceki;
    baglanti_sayisi += 2;
    liste->uzunluk--;
    bellek_birak(d);
```

H08_11_cift_bagli_liste.cpp — C++

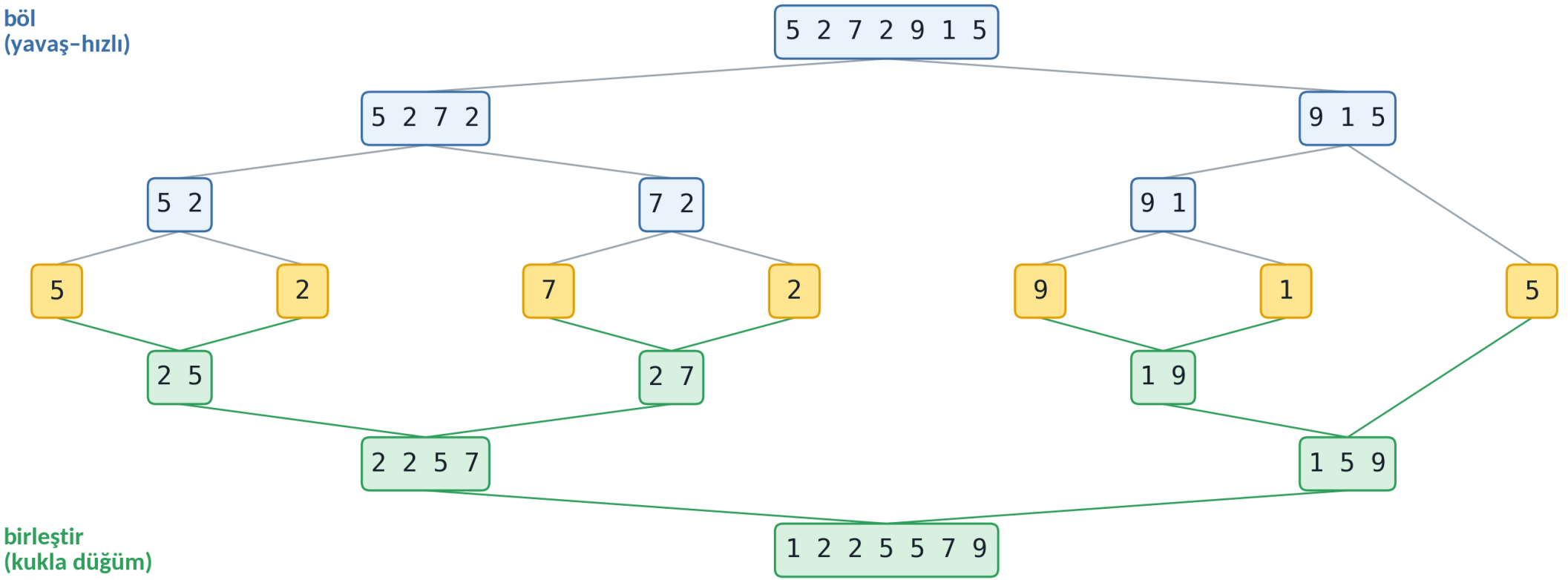
```cpp
    d->onceki->sonraki = d->sonraki;
    d->sonraki->onceki = d->onceki;
    baglanti_sayisi += 2;
    liste.uzunluk--;
    delete d;
    birakma_sayisi++;
```

H08_11_cift_bagli_liste.py — Python

```python
    d.onceki.sonraki = d.sonraki
    d.sonraki.onceki = d.onceki
    baglanti_sayisi += 2
    liste.uzunluk -= 1
    birakma_sayisi += 1     # free yok; yalnız sayaç
```

İki bağlantı, yürüme yok, özel durum yok (komşular en kötü kukla); C `free`, C++ `delete`, Python yalnızca bağlantıları koparır (satırlar: C 94–98 · C++ 72–77 · Python 74–78)

Listede birleştirmeli sıralama: böl, sırala, birleştir




```
veri#kimlik: 5#1 2#2 7#3 2#4 9#5 1#6 5#7
→ 1#6 2#2 2#4 5#1 5#7 7#3 9#5  (kararlı)
```

n = 7, veri#kimlik: eşitlerin sırası korunur · n = 10^5: 1 536 388 karşılaştırma, derinlik 18 = ⌈lg n⌉ + 1, ek dizi yok

Listede birleştirmeli sıralama: sözde kod ve C

```
algoritma SIRALA(bas)
  eğer bas = NULL veya
     bas.sonraki = NULL ise
    döndür bas
  ikinci ← ORTADAN-BÖL(bas)
  sol ← SIRALA(bas)
  sağ ← SIRALA(ikinci)
  döndür BİRLEŞTİR(sol, sağ)
```

c/H08_12_listede_birlestirmeli.c · satır 137–140 — C

```c
    Dugum *ikinci = ortadan_bol(bas);
    Dugum *sol = birlestirmeli_sirala(bas, derinlik + 1);
    Dugum *sag = birlestirmeli_sirala(ikinci, derinlik + 1);
    return birlestir(sol, sag);
```

Θ(n log n) her girdide · O(log n) yığın · kararlı · yeni düğüm 0

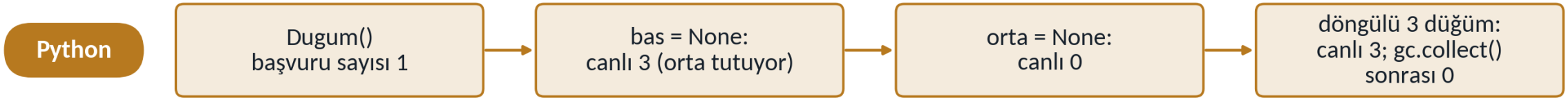
Bellek sahipliği: kim ayırır, kim bırakır?



sahip: çağıran; bırakmak onun görevi (sözleşme yorumda)



sahip: nesne; yıkıcı otomatik; ham işaretçi sahip değildir



sahip yok: başvuru sayımı + döngüsel çöp toplayıcı; sarkan başvuru olmaz

C: free sonrası işaretçi sarkar · C++: kapsam sonunda yıkıcı (RAII), unique_ptr tek sahip · Python: başvuru sayacı + döngüsel çöp toplayıcı

C: listeyi yok etmek, sonraki free'den önce

c/H08_06_liste_yok_etme.c · satır 61–70

C

```c
61  static void liste_yok_et(Dugum **bas)
62  {
63      Dugum *d = *bas;
64      while (d != NULL) {
65          Dugum *sonraki = d->sonraki;
66          dugum_birak(d);
67          d = sonraki;
68      }
69      *bas = NULL;
70  }
```

- **Sözleşme yorumda:** “dönen liste çağıranındır; liste_yok_et ile bırakılır”.
- `sonraki` free'den **önce** saklanır (tersi: Hata Avcısı 3).
- Çağıranın başı NULL yapılır: sarkan baş kalmaz.
- Derin kopya = ikinci sahip; sığ kopyayı iki kez bırakmak = çift free.

Bir ASan raporunu okumak

```
==PID==ERROR: AddressSanitizer: heap-use-after-free on address 0x… a
      pc 0x… bp 0x… sp 0x…
READ of size 8 at 0x… thread T0
    #0 0x… in liste_yok_et H08_HA3_free_sonrasi.c:34
    #1 0x… in main H08_HA3_free_sonrasi.c:48

0x… is located 8 bytes inside of 16-byte region [0x…,0x…)
freed by thread T0 here:
    #0 0x… in free asan_malloc_linux.cpp:52
    #1 0x… in liste_yok_et H08_HA3_free_sonrasi.c:35
    #2 0x… in main H08_HA3_free_sonrasi.c:48

previously allocated by thread T0 here:
    #0 0x… in malloc asan_malloc_linux.cpp:69
    #1 0x… in dugum_olustur H08_HA3_free_sonrasi.c:20
    #2 0x… in main H08_HA3_free_sonrasi.c:45

SUMMARY: AddressSanitizer: heap-use-after-free
      H08_HA3_free_sonrasi.c:34 in liste_yok_et
```

① hata türü: serbest bırakılmış öbek bloğu kullanıldı

② erişim: 8 bayt OKUMA (p->sonraki alanı)

③ nerede: satır 34 (döngü artırımı p = p->sonraki)

④ kim bıraktı: aynı fonksiyon, satır 35 (free(p))

⑤ kim ayırdı: dugum_olustur, satır 20

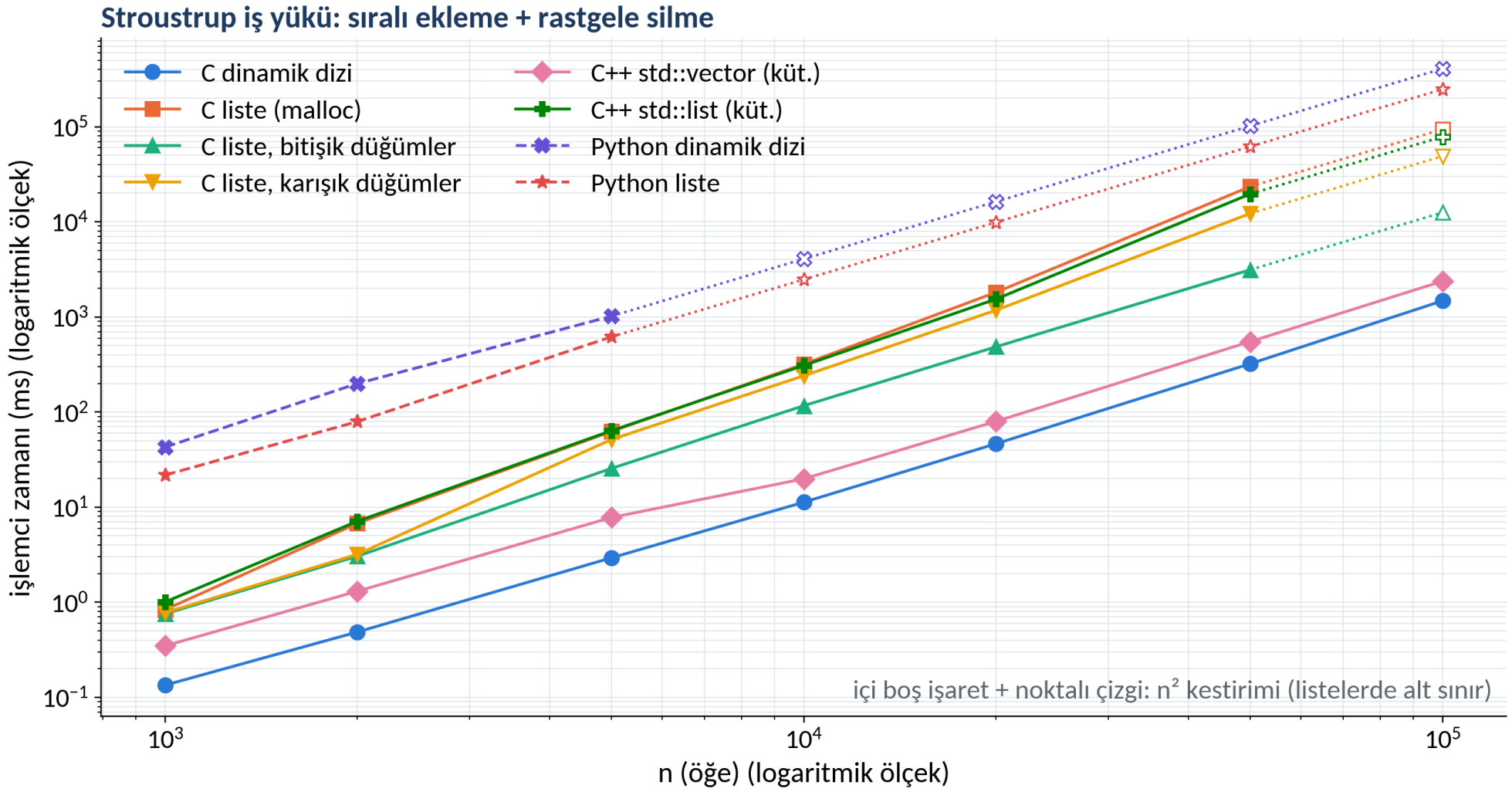
⑥ özet: tür + dosya:satır + fonksiyon

Ne oldu (türü) · nerede (ilk kendi çerçeveniz) · blok kimindi (freed by / allocated by) · özet satırı

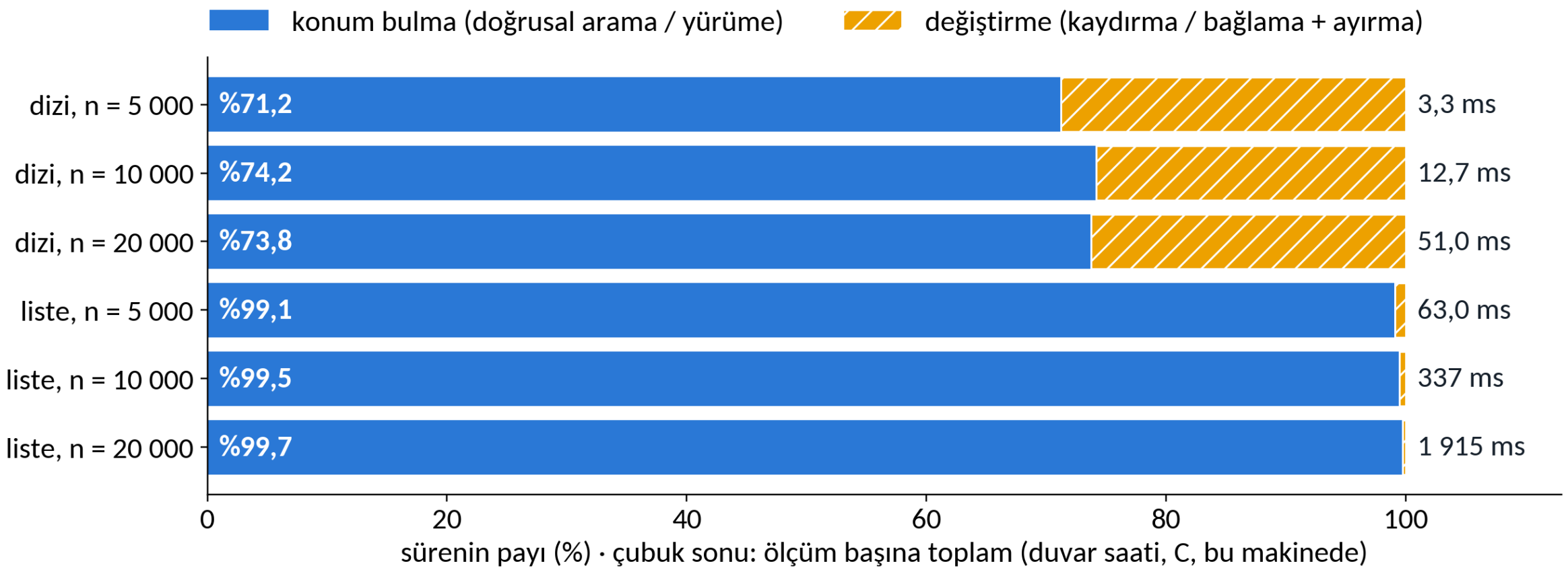
Haftanın Deneyi: Stroustrup iş yükünü ölçmek

- **İş yükü:** n rastgele tamsayı sıralı yapıya **doğrusal aramayla** eklenir, sonra rastgele konumlardan silinir (Bentley'nin alıştırması).
- **Seriler:** C dizi (× 2 realloc) · liste (düğüm başına malloc) · liste_bitisik (ideal yerleşim) · liste_karisik; C++ std::vector · std::list; Python elle dizi · elle liste · list.
- **Ölçüt:** işlemci zamanı ortancası (1 ısınma + 5 tekrar); † tek çalıştırma, (k) n^2 kestirimi.
- **Kâhin:** silinen değerlerin toplamı her seride aynı; **Örnek 13** aynı iş yükünün makineden bağımsız sayaçlarını verir. Ayrıca büyüme stratejileri ve düğüm dolaşması.

C ve C++'ta ölçülen her n'de dizi daha hızlı

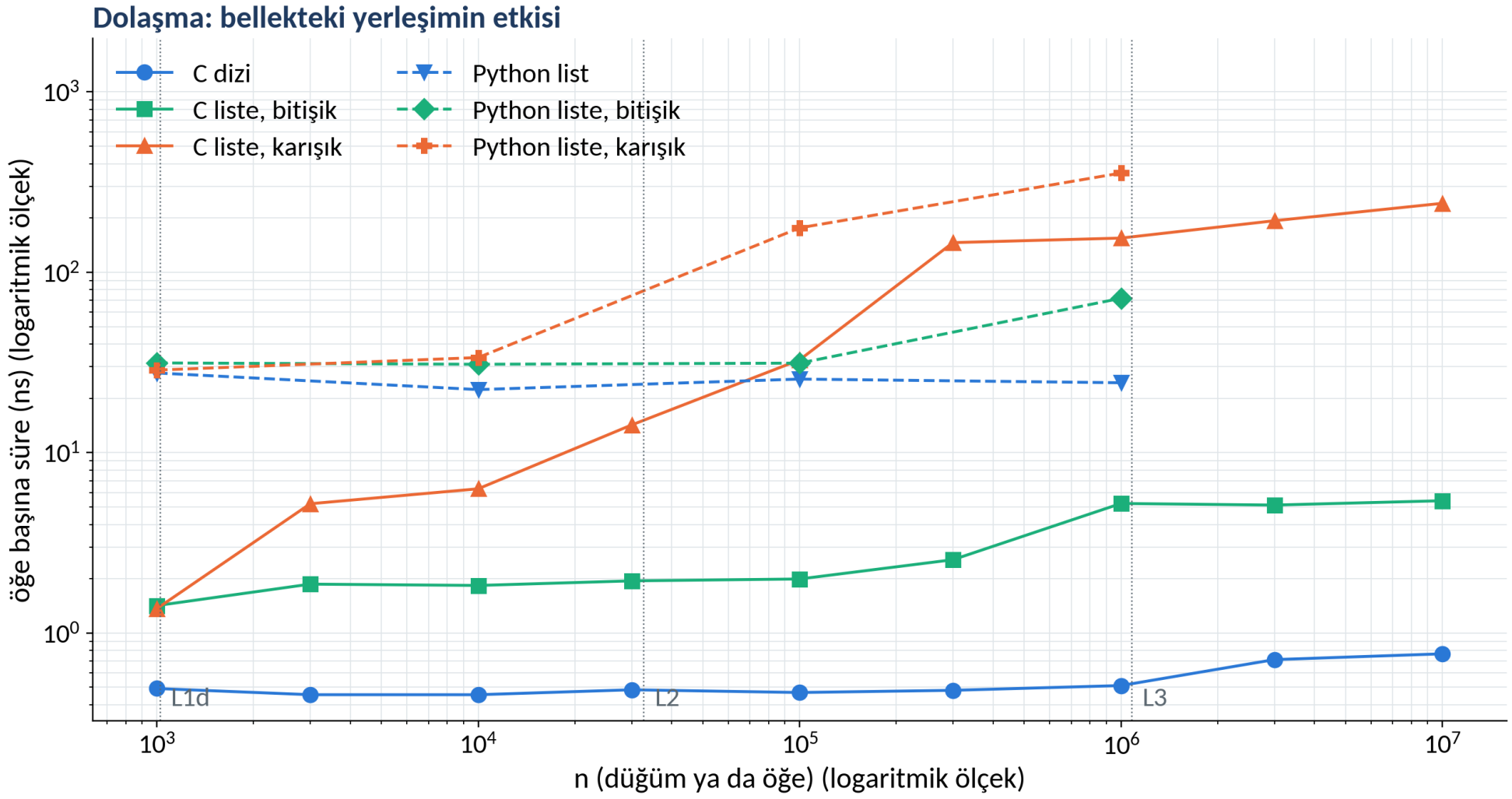


Süre nereye gidiyor? Konum bulma ve değiştirme



Konum bulmanın payı: dizide %71,2–74,2, listede %99,1–99,7 · doğrusal arama, "O(1) ekleme"yi gölgeler

Önbellek etkisi tek başına: aynı düğümler, farklı sıra



Ölçmeden genelleme yok: iki sürpriz

0,51

Python, n = 10^3: elle liste / elle dinamik dizi (liste daha hızlı)

0,25

Python list.insert/pop / elle dizi (C'de yazılmış kaydırma)

16,4 ns

C realloc, +1 büyütme, 10^6 ekleme: ekleme başına (ölçülen tekrarlarda 0 taşıma)

- Elle kopyalayan +1 dizi 10^5 eklemede 7,36 s sürdü ($\Theta(n^2)$); glibc'nin realloc'u ölçülen tekrarlarda bloğu taşımadı. Yanıt dile, gerçekleştirime ve ayırıcıya bağlı: önce ölç.

Hata Avcısı (1/2): sınır, sızıntı, sarkan işaretçi

#	Hata	Belirti (bu makinede)	Çözüm
1	Boş/tek düğüm durumunu atlamak	C: SIGSEGV; Python: AttributeError	Önce boş ve tek düğüm; n = 0, 1, 2 ile sına
2	Silinen düğümü free etmeden koparmak	Çıktı doğru; LSan: 16 bayt, 1 blok sızdı	Sakla, çıkar, sonra free
3	free sonrası p->sonraki	SIGSEGV; ASan: heap-use-after-free	sonraki'yi free'den önce sakla
4	Baş işaretçisini değerle aktarmak	C ve Python: liste [NULL]; C'de 3 blok sızar	Dugum ** ya da yeni başı döndür
5	p = realloc(p, …)	Başarısızlıkta eski blok kaybolur (LSan)	Geçici işaretçi

Hata Avcısı (2/2): bağlantı kaybı, list, döngü, yığın

#	Hata	Belirti (bu makinede)	Çözüm
6	Ters çevirmede sonraki'yi kaydetmemek	ters: [1 -> NULL]; 4 düğüm sızar	Kaydet, çevir, ilerle
7	Python list'ini bağlı liste sanmak	insert(0, x): n iki katına, süre 3,3 / 3,9 katı	deque.appendleft ya da append + reverse
8	Döngülü listeyi kırmadan yok etmek	double free, SIGABRT	Floyd ile bul, döngüyü kır, sonra yok et
9	C++ unique_ptr zincirinin varsayılan yıkıcısı	$n = 10^6$: SIGSEGV (yığın taşması)	Yıkıcıda döngü: bas = std::move(bas->sonraki)

Dil Penceresi: bellek ve bağlı yapılar

Öğe	C	C++	Python
Bellek yönetimi	malloc/free elle	RAII, unique_ptr	başvuru sayımı + gc
Boş bağlantı	NULL	nullptr	None (is None)
Düğüm kimliği	p == q	p == q	p is q
Dinamik dizi	elle + realloc	std::vector	list
Bağlı liste	elle	std::list, std::forward_list	yok; collections.deque
Büyüme	seçim sizde	gerçekleştirime bağlı (× 2; 21 değişim)	fazladan ayırma: (n + n/8 + 6) & ~3
Bellek gözlemi	sizeof(Dugum) = 16	sizeof, capacity()	sys.getsizeof, __slots__

Mühendis gözüyle ve merak kutusu

Mühendis gözüyle

- **Önce ölç:** kuramsal üstünlük önbellek gerçekliğiyle çelişebilir (Stroustrup deneyi).
- **Sahiplik arayüzde yazılır:** her ayıran fonksiyonun yorumunda kim bırakır.
- **Araçları sürekli çalıştırın:** sızıntı sessizdir; `make asan`, Valgrind.
- **Sınır durumları test vakasıdır:** n = 0, 1, 2; baş ve son.

Merak kutusu

- Düğümleri tek blokta ayırsak? $n = 5 \cdot 10^4$'te malloc'lu liste 23,45 s, tek bloklu sıralı liste 3,13 s: **havuz (arena) ayırıcı**.
- **Açılmış bağlı liste:** düğüm başına birkaç öğe. Konum bulma kaç önbellek satırına dokunur?
- Dizinin 321,7 ms'sine ne kadar yaklaşılır? Kendi programınızla ölçün.



MİNİ-OBS 2 PROJESİ

Mini-OBS 2 v2.7: bekleme listesi ve bağlı listede birleştirme

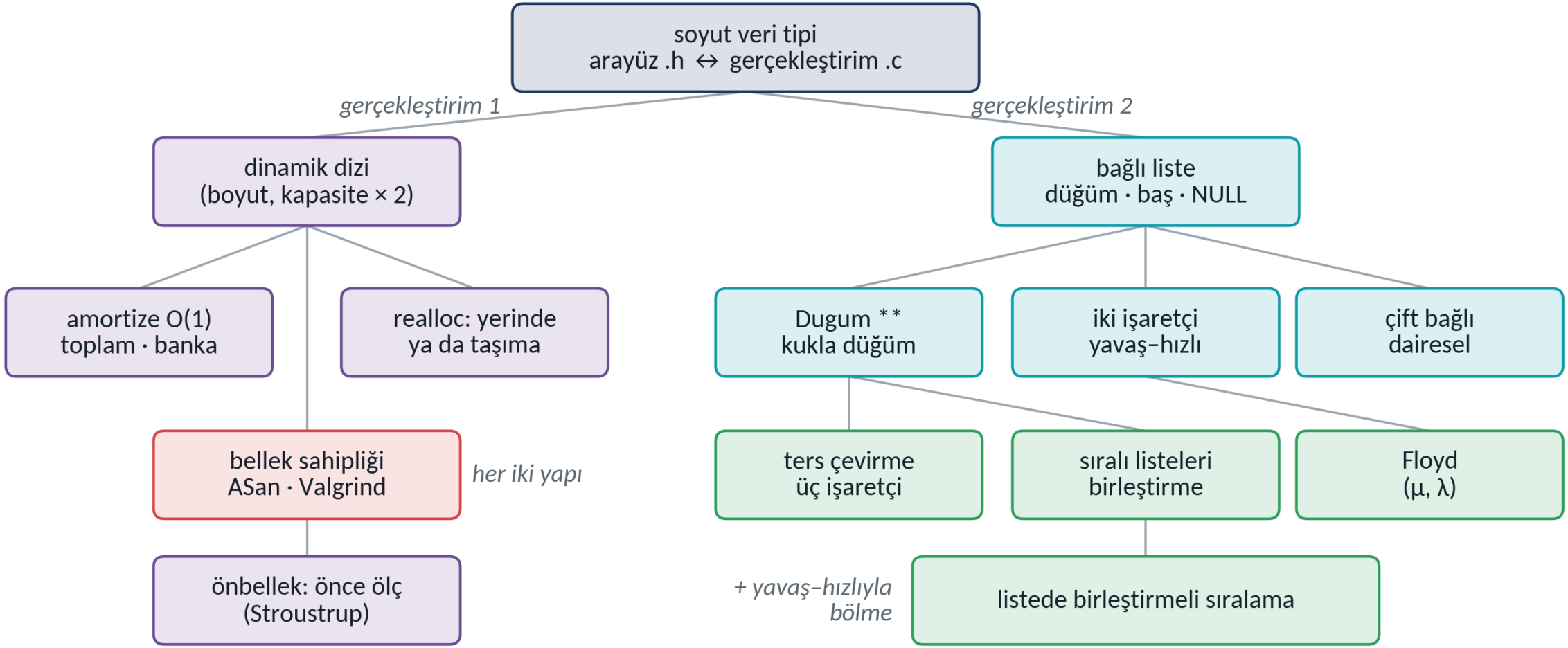
- `bekleme.h/.c`: kuyruk işaretçili tek bağlı liste; sona ekle O(1), baştan çıkar O(1), numarayla sil `BeklemeDugumu **` ile.
- `kontenjan.c`: kontenjan açılınca bekleyen ilk öğrenci **otomatik aktarılır**.
- `kayit_listesi.c`: iki şubenin sıralı listeleri **kukla düğümle, düğüm taşımadan** birleşir.
- `dinamik.c`: × 2 büyüme + kopyalama sayacı; sahiplik tablosu; `make asan` ve `make valgrind` temiz.
- **Ara proje teslimi: H8'in ilk dersinin başı** (v2.6 + ölçüm raporu; yönerge § 11 denetim listesi).
- **Atölyeler:** kutu–ok izleme (altı işlem) · liste kütüphanesi ve birim testleri (eşli) · sızıntı avı (ASan + Valgrind).

İLERİ BAKIŞ

Bu hafta nerede derinleşiyor?

- **H9:** yığın ve kuyruğun bağlı liste gerçekleştirimi (başa ekleme, baştan çıkarma, kuyruk işaretçisi) ve dizi gerçekleştirimi (dairesel tampon).
- **H10:** zincirlemeli karma tablosunda her kova bir bağlı liste; tablo büyütmenin amortize maliyeti bu haftanın analizi.
- **Veri Yapıları:** ağaçlar (BST, AVL), atlamalı listeler, graf temsilleri bağlı yapılar üzerine kurulur.
- **İşletim Sistemleri:** malloc'un iç yapısı, sayfalar ve önbellek.
- **Hazırlık:** Bölüm 11'deki alıştırmalar ve web sayfasındaki oynatıcılar; H9'un sorusu: hesap makinesi 3 + 4 × 2'de önce çarpmayı yapması gerektiğini nereden bilir?

Özet: kavram haritası



ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1. × 2 büyüyen ve ilk kapasitesi 1 olan dinamik diziye **1 000** öğe eklenince toplam kaç öğe kopyalanır?

2. `Dugum **p` neyi gösterir? Baş düğüm silinirken neden ayrı bir dal gerekmez?

3. Stroustrup iş yükünde listenin süresinin büyük kısmı **nereye** gider ve neden?